



Understanding Static and Non-Static Constructors in C#

By Albert Harmon Published September 30, 2017 ⌚ 3 min read [Education](#)



TLDR This blog post explores the differences between static and non-static constructors in C#, detailing their definitions, execution behavior, initialization responsibilities, and limitations. It provides clear examples and explanations to help developers understand when and how to use each type of constructor effectively.

In this blog post, we will delve into the differences between static constructors and non-static constructors in C#. This discussion builds upon previous parts of our series on constructors, where we covered the types and uses of constructors.

What is a Constructor?

A constructor is a special method used to initialize objects of a class. It is called when an instance of the class is created. Constructors can be

categorized into two main types: static constructors and non-static constructors.

Static Constructors

A static constructor is explicitly declared using the ``static`` modifier. Here are some key points about static constructors:

- **Initialization:** Static constructors are responsible for initializing static fields of a class.
- **Execution:** Static constructors are implicitly called when the class is first accessed. They execute immediately once the execution of the class starts and are the first block of code to run within the class.
- **No Parameters:** Static constructors cannot have parameters. This is because they are called implicitly, and there is no way to pass parameters to them.
- **Single Execution:** A static constructor executes only once during the lifecycle of the class, regardless of how many instances are created.

Example of a Static Constructor

```
class ConstructorsDemo
{
    static ConstructorsDemo()
    {
        Console.WriteLine("Static constructor is called");
    }
}
```



Non-Static Constructors

Non-static constructors, also known as instance constructors, are used to initialize instance fields of a class. Here are some important aspects of non-static constructors:

- **Initialization:** Non-static constructors initialize non-static fields of a class.

- **Execution:** Non-static constructors must be explicitly called when creating an instance of the class. They execute each time an instance is created.
- **Parameters:** Non-static constructors can be parameterized, allowing values to be passed during instantiation.
- **Overloading:** Non-static constructors can be overloaded, meaning multiple constructors can exist with different parameters.

Example of a Non-Static Constructor

```
class ConstructorsDemo
{
    public int x;
    public ConstructorsDemo(int value)
    {
        x = value;
        Console.WriteLine("Non-static constructor is called");
    }
}
```

```
}  
  
}
```

Key Differences Between Static and Non-Static Constructors

Feature	Static Constructor	Non-Static Constructor
---------	--------------------	------------------------

Declaration	Declared with <code>static</code> modifier	No modifier required
Initialization Responsibility	Initializes static fields	Initializes instance fields
Execution	Implicitly called	Must be explicitly called
Execution Timing	Executes once when the class is accessed	Executes each time an instance is created
Parameters	Cannot have parameters	Can have parameters
Overloading	Cannot be overloaded	Can be overloaded

| Implicit Constructor | Implicitly defined if static fields exist | Implicitly defined unless explicitly defined |

Conclusion

Understanding the differences between static and non-static constructors is crucial for effective class design in C#. Static constructors are useful for initializing static data and ensuring that certain actions occur only once, while non-static constructors are essential for setting up instance-specific data. By knowing when to use each type, developers can create more efficient and organized code.

Thank you for reading! For more insights and tutorials, stay tuned to our blog.